Algorithms – Final Exam

2.2.26 – term A

Prof. Tami Tamir

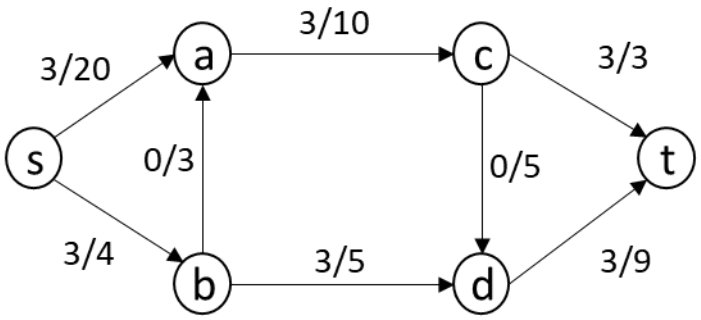
- Exam length: 3 hours
- Allowed open material: Attached fact-sheets + one paper brought by the student.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- You may use scratch paper, but answers must be written only on the exam form.
- Note that only the front side of each page is scanned for grading. If you need more space, use the last page and clearly direct the grader there.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

G O O D L U C K !

♣ Question 1 (18 pts.)

♣ **1.1** (**8 pts**.) The figure presents a flow network with a legal flow. Every edge e∈E is labeled by f(e)/c(e) . It is known that the given flow is a result of **two** iterations of Ford & Fulkerson algorithm.

No need to justify your answers in this section.



1. What was the augmenting path in the first iteration? s-b-a-c-d-t (or s-b-a-c-t)
2. What was the augmenting path in the second iteration? s-a-b-d-c-t (or s-a-b-d-t)
3. What is the value of the maximum flow in the network? 12
4. Write all sets of nodes that define a min-cut in the network:

{s,a,c}, {s,a,b,c,d}

♣ **1.2** (10 pts.) Let G = (V, E) be a flow network with source s and sink t.

Definition: A node v is **upgradable in G** if $(s, v) \notin E$, and adding an edge (s, v) with a positive capacity to E creates a network where the max-flow value is higher than the max-flow value in G.

Propose an efficient algorithm that returns the set of nodes U such that $v \in U$ if and only if v is upgradable in G.

Justify the algorithm's correctness and analyze its time complexity.

a. Run Edmonds-Karp to find a max flow in G
b. Compute the set T of vertices that can reach t in residual network of G w.r.t. max flow (Run BFS(t) of the last residual network with reverse edge directions)
c. For every v in T, if (s,v) is not in E, add v to U.

Justification: v is upgradable iff adding an edge (s,v) with positive capacity creates a new augmenting path in the last residual network.

Time Complexity:
a. computing max flow using Edmonds-Karp alg.: $O(|E|^2 |V|)$
b. Constructing the reversed graph and BFS: $O(|E|)$
c. $O(|V|)$, sunce $|T|=O(|V|)$
Total: $O(|E|^2|V|)$

Question 2 (15 pts.)

Let G = (V, E) be a simple directed acyclic graph (**DAG**). Let s∈V be a **root** of G. Let $G^r = (V, E^r)$ be the graph obtained by reversing the edge directions in G. That is, $E^r = \{(v, u) \mid (u, v) \in E\}$.
Prove or Disprove each of the following claims:

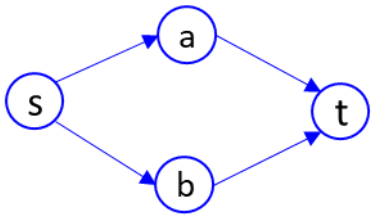
♣**2.1.** (4 pts.) The node **s** is necessarily the last node in any topological sort of G^r.

Proof: Assume by contradiction that u comes after s in a topological sort of G^r. It means that there is no path from u to s in Gr, thus, there is no path from s to u in G. Contradicting the fact that s is a root in G.

♣ **2.2** (5 pts.) If G^r has a root, then executing BFS(s) on G results in a directed spanning tree where out-degree(s) = 1.

Counter example: t is a root in Gr.

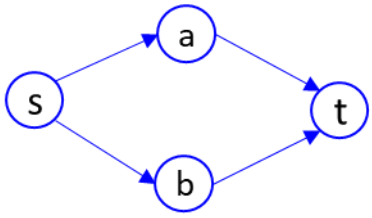
BFS(s) includes the edges {sa,sb,at}



♣ **2.3** (5 pts.) If G^r has a root, then executing DFS(s) on G results in a directed spanning tree where out-degree(s) = 1.

Counter example: t is a root in Gr.

DFS(s) includes the edges {sa,sb,at}



Another counter example for both 2.2 and 2.3: A graph with a single node.

Question 3 (18 points)

N electric vehicles arrive at a charging station with two chargers: one slow and one fast. For each vehicle j, two parameters $0 < f_j < s_j$ are given: s_j is the charging time (in minutes) of vehicle j on the slow charger, and f_j is the charging time of vehicle j on the fast charger. All vehicles must be charged and the charging cannot be split between the two chargers.

Propose a **Dynamic Programming** algorithm to determine if all **N** vehicles can be charged within at most **T** minutes. All input values are integers.

For $0 \le j \le N$, $0 \le t_1 \le T$, $0 \le t_2 \le T$, let **Charge[j, t_1, t_2]** be a Boolean variable whose value is True if and only if the first j vehicles can be charged such that the slow charger is active for at most t_1 minutes and the fast charger is active for at most t_2 minutes.

3.1 (2 pts.) What is the expression describing the solution's value (no need to explain)?

Charge[N,T,T]

♣ **3.2** (12 pts.) Write a dynamic programming formula for calculating **Charge[j, t_1, t_2]**. Distinguish between the base and the general case. Explain the formula in words.

Base case: Charge[0,t1,t2]=True for all $0 \le t_1 \le T$, $0 \le t_2 \le T$. Also add a wrapping Charge[j,t1,t2]=False for all j and t1<0 or t2<2.
General DP formula:
For every j>0, Charge[j,t1,t2]= Charge[j-1,t1-sj,t2] OR Charge[j-1,t1,t2-fj].

Explanation: The charging time of 0 cars is zero, and it is not possible to do anything in a negative length time interval.
If the j-th car is charged on the slow machine, then the first j-1 cars are charged in time t1-sj on the slow machine and time t2 on the fast machine. If the j-th car is charged on the fast machine, then the first j-1 cars are charged in time t1 on the slow machine and time t2-fj on the fast machine.

♣ **3.3** (4 pts.) Describe the DP table, its size, and the order of filling. Analyze time complexity as a function of N and T. Justify your answer.

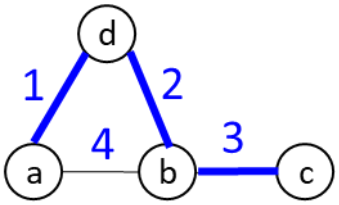
A 3-dim table having depth N and slices of size T*T ($0 \le j \le N$,$0 \le t1 \le T$, $0 \le t2 \le T$)
The base case corresponds to the first slice (j=0).
For the calculation of Charge [j,t1,t2] we need entries from slice j-1, therefore the table can be filled slice after slice. Each entry takes O(1) for a total of $O(NT^2)$.

Question 4 (18 points)

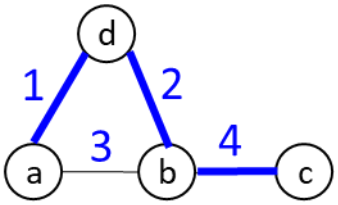
Let G = (V, E) be simple, connected, and undirected graph. Let S be a set of |E| distinct positive numbers, corresponding to costs. An assignment of S to E is a one-to-one mapping from S to E, such that each value in S is set as the cost of exactly one edge.

Note that an assignment of S to E results in a weighted graph with costs on the edges.

4.1. Warm-up (2 pts): For the graph in the figure and S = {3, 2, 4, 1}, describe an assignment of S to E that fulfills the description below each figure. Indicate the costs next to the edges and highlight the edges of the minimum spanning tree.



The cost of a Minimum Spanning Tree in the resulting weighted graph is 6



The cost of a Minimum Spanning Tree in the resulting weighted graph is 7

♣ **4.2** (8 pts.) Given G=(V,E) and S, suggest an efficient algorithm that determines an assignment of S to E, such that the cost of a **minimum** spanning tree in the resulting weighted graph is the **minimal** possible.
Prove the correctness of your algorithm and analyze its time complexity.

1. Run BFS or DFS to compute any spanning tree of G. T=(V,E_T).
2. Find the (n-1)-th smallest cost in S [More efficient than sorting]
3. Let S0 be the set of the n-1 smallest numbers in S. Let S1 be the set of the m-n+1 largest numbers in S.
4. Assign S0 to E_T, arbitrarily. Assign S1 to E-E_T, arbitrarily.

Justification: Since the MST includes n-1 edges, we cannot have a ST whose cost is lower than the sum of the cheapest n-1 costs in S. The algorithm achieves this bound.

Time complexity:
Step 1: O(|E|)
Step 2: O(|E|) – using an efficient select algorithm
Step 3: O(|E|) – scan S and partition into the two sets.
Step 4: O(|E|) – O(1) per edge.
Total: O(|E|)

♣ **4.3** (8 pts.) Given a graph G=(V,E) where |V|=|E|, and a set S, suggest an efficient algorithm to determine an assignment of S to E, such that the cost of a **minimum** spanning tree in the resulting weighted graph is the **maximal** possible. Prove the correctness of your algorithm and analyze its time complexity.

1. Run BFS or DFS to compute any spanning tree of G. T=(V,E_T).
2. Let (u,v) be the remaining edge. It closes a cycle in the tree. Find all edges in this cycle by moving from u to v in the shortest path route on the spanning tree.
3. Let C be the set of edges in the cycle. Find the |C|-th smallest cost in S.
4. Let S0 be the set of the |C| smallest numbers in S. Let S1 be the set of the m-|C|+1 largest numbers in S.
5. Assign S0 to the edges of C, arbitrarily. Assign S1 to E-C, arbitrarily.

Justification: Since |V|=|E|, the graph includes one cycle. Every ST includes all edges except for the most expensive edge in the cycle. The algorithm assigns the minimal costs to the cycle edges. This way, the cost of the MST will be the total sum of S minus the |C|-th value. This is the maximal possible, because any MST will include the cheapest |C|-1 edges.

Time complexity:
Step 1: O(|E|)
Step 2: O(|E|)
Step 3: O(|E|) – using an efficient select algorithm
Step 4: O(|E|) – scan S and partition into the two sets.
Step 5: O(|E|) – O(1) per edge.
Total: O(|E|)

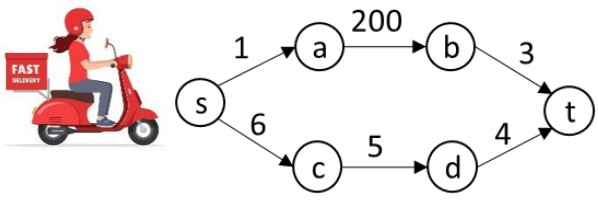
♣ **Question 5 (18 points)**

Given is a directed graph $G = (V, E)$ with positive edge weights given by $c: E \to \mathbb{R}^+$, and a source node $s \in V$. The graph represents a road network and the weights denote the travel time on the corresponding roads. A delivery company has developed a special booster installed on all its scooters. The booster can be activated at most once per trip.
For a path p=v_0-v_1-…-v_k, if the booster is activated at vertex v_i, the scooters 'takes off' and 'lands' at vertex v_{i+2} after 0 time units, regardless of the weights of the edges (v_i, v_{i+1}) and (v_{i+1}, v_{i+2}).

Suggest an efficient algorithm that finds for every $v \in V$ the minimum travel time from s to v. Describe the algorithm in words and formally, justify its correctness (state the main claim and justify it shortly), and analyze its time complexity.

Clarification: The booster cannot be used for a single edge. In a path p=v_0-v_1-…-v_k, the booster can only be used if k≥ 2, and only once, at a vertex v_i where 0≤i≤k-2.

Example; In the graph in the figure d(s,s)=0, d(s,a)=1, d(s,b)=0, d(s,c)=6, d(s,d)=0. For node t, the cost of the path s-a-b-t is 1 (the booster is activated at node a), and the cost of the path s-c-d-t is 4 (the booster is activated at node s), therefore, d(s,t)=1.



Write your answer in the next page.

Answer to Question 5

Algorithm:

1. Let G'=(V',E'), $V'=V^1 \cup V^2 \cup V^3$ where each of V^i is a copy of V.
   $E'= \{(u^1,v^1)|(u,v) \in E\} \cup \{(u^3,v^3)|(u,v) \in E\}$ - with c'(e)=c(e).
   $\cup \{(u^1,v^2)|(u,v) \in E\} \cup \{(u2,v^3)|(u,v) \in E\}$ - with c'(e)=0.
2. Run Dijkstra on G' with s^1 as a source.
3. For every v in V, return min($d(s^1,v^1)$, $d(s^1,v3)$).

In words: There are three copies of the vertices. For every edge (u, v) in E with weight w, we add to G' edges (u1, v1) and (u3, v3) with weight w, an edge (u1, v2) with weight 0, corresponding to the first part of a jump, and an edge (u2,v3) with weight 0, corresponding to the second part of a jump.

Correctness: In G'

- There is a path from s1 to v1 that has cost k, iff there is a path from s to v in G that does not use the booster and has cost k.
- There is a path from s1to v3 that has cost k, iff there is a path from s to v in G that has cost k+x and includes as a subpath two edges e1-e2 whose total cost is x.

Main idea: Paths in G are preserved in the first copy. Paths that use the booster lead to the third copy. There are no internal edges in the second copy. The first edge in the jump leads from V1 to V2 and the second edge of a jump leads from V2 to V3.

Time complexity:
Step 1: O(|V|+|E|). G' has 3|V| nodes and 4|E| edges.
Step 2: O(|E'|+|V'|log|V'|) = O(|E|+|V|log|V|) assuming implementation of Dijkstra with Fib. heaps.
Step 3: O(|V|)

All together: O(|E|+|V|log|V|)

Question 6 (14 points)

n ice cubes are in a freezer. The volume of the i-th cube is v_i. Noa has a water pitcher, and at any given moment, there must be one ice cube in it. Noa takes the cubes out of the freezer one by one and places them in the pitcher. An ice cube placed in the pitcher with a volume of x melts within x time units.

Each time Noa places a cube in the pitcher, she waits until it melts completely before inserting the next one. Every time Noa opens the freezer to take out a cube, all the cubes (including the one she is currently removing) melt slightly and lose 1% of their original volume. This means that if a certain cube has an initial volume v_i and it is the k-th cube Noa places in the pitcher (k=1,…, n), its volume upon entering the pitcher is $v_i(1-0.01k)$. It can be assumed that $n < 100$, so no cube will melt completely inside the freezer.

Example: Suppose there are two cubes with $v_1=100$ and $v_2=400$.

- If cube 1 is inserted first, it melts within 100(1-0.01)=99 time units. When cube 2 is inserted, its remaining volume is 400(1-0.02)=392; therefore, it will be in the pitcher from time 99 until time 491.
- If cube 2 is inserted first, it melts within 400(1-0.01)=396 time units. When cube 1 is inserted, its remaining volume is 100(1-0.02)=98; therefore, it will be in the pitcher from time 396 until time 494.

♣ **6.1 (7 pts)** Suggest a greedy algorithm that determines the order of inserting the ice cubes into the pitcher such that the total duration of time there is an ice cube in the pitcher is maximized. Describe the algorithm and analyze its running time. You can assume that all v_i values are known in advance.

Algorithm:

1. sort the cubes such that $v_1 \geq v_2 \geq \ldots \geq v_n$.
2. Insert the cubes to the pitcher according to the sorted order (large cube first).

Time complexity: O(n log n) for the sorting

♣ **6.2 (7 pts) State** and **prove** the Greedy Choice Property suitable for proving the correctness of your algorithm.

Greedy Choice Property (GCP): There exists an optimal order in which the ice cube v_1, (with maximal v_i), is first.

Proof: Let A be any order of the ice cubes. Assume that the largest cube is not first. Specifically, assume it is in the k-th position in the sequence for k>1. Let j>1 be the ice cube in position 1. Consider a solution in which cube 1 and cube k swap.

The melting time for all other ice cubes remain unchanged, as their positions in the sequence does not change. The total melting time of cubes 1 and j before the swap is $v_j(1-0.01)+v_1(1-0.01k)$. Their total melting time after the swap is $v_1(1-0.01)+vj(1-0.01k)$.

The difference (total after – total before) is $vj(1-0.01k)+v1(1-0.01)- v_1(1-0.01k)-vj(1-0.01) = v_1(0.01k-0.01)-v_j(0.01k-0.01)$, which is non negative for every $v_1 \geq vj$.